

CMP-142 Object Oriented Programming

Lab-10

Topics:

- Exception Handling

Instructions!

- This is a **graded lab**, you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed to ask how he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or instructor.
- Indent your code properly.
- Use meaningful variable and function names. **Use the camelCase notation.**
- Save your work frequently. Make a habit of pressing CTRL+S after every line of code you write.
- **Do NOT use any global or static variable(s).** However, global named constants may be used.
- Make sure that there are **NO dangling pointers or memory leaks** in your programs.
- Make sure that all functions have been implemented and can be called in the main function. Otherwise, no marks will be awarded for that function.
- **Prototypes of some functions are omitted. You are supposed to complete them on your own. Moreover, there might be a case where some functions are ignored altogether while giving structure of class. You have to write and complete them.**

[Task 1] Advanced Conference Management System with Dynamic Memory Allocation (ACMS-DMA)

[50 Marks]

Objective:

The objective of this task is to design and implement an Advanced Conference Management System (ACMS) with Dynamic Memory Allocation (DMA), simulating the roles and functionalities of various employees in a conference setting. The system should be capable of handling multiple employee roles such as Speaker, Organizer, and Attendee, while also supporting a specialized multi-role employee type that combines the responsibilities of multiple roles.

The key goals of this task are to:

1. Demonstrate the use of dynamic memory allocation for string handling, **avoiding the use of `std::vector` or `std::string` and instead utilizing `new` and `delete[]` for memory management.**
2. Employ **multiple inheritance and virtual inheritance** to model the relationships between different employee roles.
3. Use **pure virtual functions** to define common functionality across all employee roles.
4. Implement function templates to assign generic tasks to employees.

The system should be able to:

- Assign dynamic tasks to employees using function templates.
- Perform role-specific tasks for each employee based on their individual roles (Speaker, Organizer, Attendee, or MultiRoleEmployee).
- Display relevant details for each employee, such as their name, email, and tasks they are assigned to.

Class Structure:

1. Abstract Base Class:

- A base class `Employee` will be used to define common attributes such as employee ID, name, and email. This class will also define a pure virtual function `performRoleTask()` which will be overridden by derived classes.

2. Derived Classes for Specific Roles:

- **Speaker:** Represents an employee who is a speaker at the conference. Each speaker will have a session topic and a session duration. The `performRoleTask()` function will print information about the speaker's session.
- **Organizer:** Represents an employee who is responsible for organizing the conference. Each organizer will have a schedule of events. The `performRoleTask()` function will print details about the organizer's role in managing the event.
- **Attendee:** Represents an employee who attends specific sessions at the conference. Each attendee will have a list of sessions they will attend. The `performRoleTask()` function will print the sessions the attendee will attend.

3. MultiRoleEmployee:

- A class `MultiRoleEmployee` will inherit from `Speaker`, `Organizer`, and `Attendee`, demonstrating the use of multiple inheritance. It will be capable of performing all the tasks of a speaker, organizer, and attendee.
- This class will also implement a function template `assignRoleTask()` to allow for the dynamic assignment of tasks to employees.

4. Memory Management:

- For all attributes that involve strings, dynamic memory allocation will be used. Specifically, memory for `employeeName`, `email`, `sessionTopic`, `eventSchedule`,

and sessionsToAttend will be allocated using new and deallocated using delete[] to ensure efficient memory management.

Functionality:

- **Employee Details:** The system should display the employee's details, including their ID, name, email, and the specific details related to their role (e.g., session topic for a speaker, event schedule for an organizer).
- **Role-Specific Tasks:** The performRoleTask() function must be implemented in each derived class (Speaker, Organizer, Attendee) to perform tasks specific to the role.
- **Task Assignment:** The assignRoleTask() function template should be used to dynamically assign tasks to employees based on their roles. This demonstrates the ability to handle various types of tasks in a generic manner.
- **Dynamic Memory Allocation:** The system must use dynamic memory allocation to store and manage employee names, emails, session topics, event schedules, and session lists for attendees, avoiding the use of built-in containers like std::vector or std::string.

Requirements:

1. **Use of Dynamic Memory Allocation:**
 - All strings (such as employeeName, email, sessionTopic, etc.) must be allocated dynamically using new and deallocated using delete[].
2. **Pure Virtual Functions:**
 - The base class Employee must define a pure virtual function performRoleTask() which will be overridden in derived classes to implement role-specific behavior.
3. **Multiple Inheritance:**
 - The MultiRoleEmployee class should inherit from Speaker, Organizer, and Attendee, demonstrating multiple inheritance. Virtual inheritance should be used where appropriate to prevent ambiguity.
4. **Template Functions:**
 - The assignRoleTask() function template should allow dynamic task assignment, and the system should print the task assigned to each employee.

Expected Output:

Employee ID: 1
Employee Name: Alice Smith
Email: alice@tech.com
Session Topic: AI in Healthcare
Session Time: 30 minutes
Event Schedule: 9:00 AM - Opening, 9:30 AM - Break, 10:00 AM - Workshop
Sessions to Attend: AI for Good, Tech Innovations
Alice Smith is presenting the session: AI in Healthcare for 30 minutes.
Alice Smith is organizing the event with schedule: 9:00 AM - Opening, 9:30 AM - Break, 10:00 AM - Workshop
Alice Smith is attending the session(s): AI for Good, Tech Innovations
Assigning Task: Coordinate Q&A session for 'AI in Healthcare'

Employee ID: 2
Employee Name: Bob Johnson
Email: bob@tech.com
Session Topic: Blockchain in Finance
Session Time: 45 minutes
Event Schedule: 10:00 AM - Opening, 11:00 AM - Break, 12:00 PM - Panel
Sessions to Attend: Blockchain for Security, Digital Finance
Bob Johnson is presenting the session: Blockchain in Finance for 45 minutes.
Bob Johnson is organizing the event with schedule: 10:00 AM - Opening, 11:00 AM - Break, 12:00 PM - Panel
Bob Johnson is attending the session(s): Blockchain for Security, Digital Finance
Assigning Task: Host panel discussion on 'Blockchain in Finance'